

Neural networks - Project 1

MLP regression

Martin Mislovič, Hana Magyarová, Lenka Husárová

Last updated: March 26, 2025

✓ Task

Our task was to predict house prices using MLP with Adam optimizer. This is a regression problem.

✓ Metrics

Since this is a regression problem, we agreed to use following metrics:

- Mean Squared Error (MSE) - we also used it as *loss function*
- Mean Absolute Error (MAE)
- Root Mean Squared Error (RMSE)

✓ Codes

- **eda_preprocessing.ipynb** - notebook with exploratory data analysis and preprocessing. TLDR; we removed some outliers, scaled and transformed our data and performed feature selection.
- **model.ipynb** - notebook with our model, its training and experiments
- **sweep configs** - we created 8 *.yaml* configuration files in total. Each of them has different parameters changed, as their name suggests.
 - **activation.yaml**
 - **batch_size.yaml**

- dropout.yaml
- hidden_layers.yaml
- learning_rate.yaml
- optim_dropout.yaml
- optim_no_dropout.yaml
- best.yaml

✓ Dataset

[Available here.](#)

In the end, we did not drop any columns, meaning we continued with all 9 features:

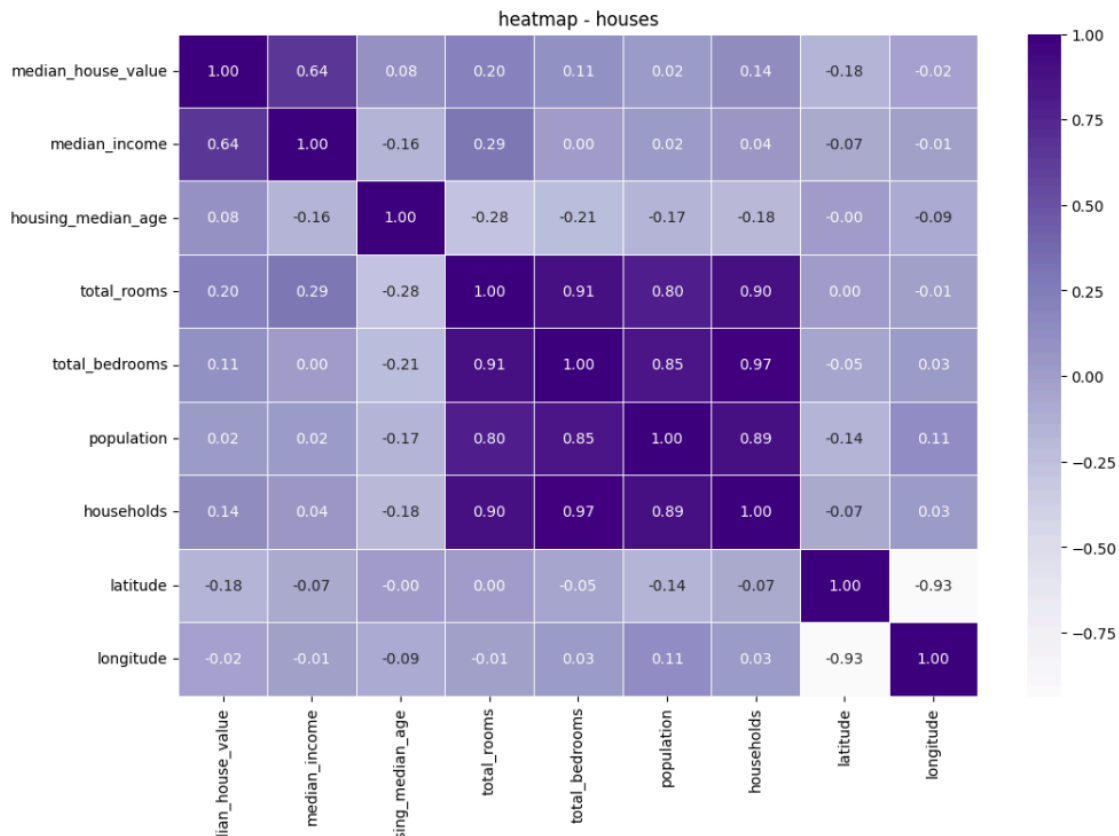
- median_house_value (target)
- median_income
- housing_median_age
- total_rooms
- total_bedrooms
- population
- households
- latitude
- longitude

As we mentioned, we dropped some of the outliers. Our sample size went down from **20640** to **17621** samples.

✓ EDA, preprocessing

First, we checked for null values and overall statistics of the dataset. Then, we continued with checking out the distribution of features. Based on this, we decided that we need to remove outliers and also transform and scale data. We also decided to scale the target variable. This way we hope to achieve better stability of the model

and improved performance, it is however important to note that this means data unprocessing (reverse transformation) must be done to the model predictions. After that, we checked again the distributions of our features and printed correlation heatmap, which helped us to determine which features could potentially be good predictors for target variable.



Correlation heatmap after data preprocessing

As a last step in preprocessing, we performed feature extraction. We used multiple techniques, but each of them gave us different results. So we decided we will keep all the predictors.

✓ Training

We started by loading our data and defining model architecture. We initialized, trained, validated and tested our model. We stucked to the MSE loss function, but we also used MAE and RMSE for evaluation. Our dataset split is 80/10/10 -> 80% for the training and 10% for the

test and validation sets. We also implemented early stopping to prevent overfitting. Results of the initial training with default parameters were:

Configuration used:

```
batch_size = 1000,  
num_workers = 0,  
activation = "leakyrelu",  
use_dropout = False,  
dropout_rate = 0.2,  
optimizer = "Adam",  
epochs = 100,  
learning_rate = 0.001,  
use_batch_norm = True,  
hidden_sizes = [128, 64, 32]
```

✓ Experiments

We performed several experiments on our model to observe how different hyperparameters affect it. We experimented with different dropout rates, numbers of hidden layers, batch sizes, learning rate and activation functions. We were able to do this thanks to sweep configurations and wandb. We described all of this in detail in the following chapters.

✓ Sweep configuration files

We created the following sweep config files:

- **activation.yaml**
- **batch_size.yaml**
- **dropout.yaml**
- **hidden_layers.yaml**
- **learning_rate.yaml**
- **optim_dropout.yaml**
- **optim_no_dropout.yaml**

Here is an example of one of the sweep config files, specifically activation.yaml:

```
project: NN_assignment1
name: Activation_funcions_test
method: grid
metric:
  name: loss_val
  goal: minimize

parameters:
  optimizer:
    values: ["Adam"]

  learning_rate:
    values: [0.01]

  batch_size:
    values: [64]

  epochs:
    min: 20
    max: 400

  hidden_sizes:
    values:
      - [4,8,16]

  activation:
    values: ["relu", "leakyrelu", "tanh", "sigmoid", "linear", "PReLU"]

  use_dropout:
    value: False

  dropout_rate:
    value: 0.5

  use_batch_norm:
    value: True
```

✓ Different dropout values

The experiment revealed there seems to be no significant difference in performance with differing dropout values. Best performance was achieved with dropout of about 23%.

✓ Different number of hidden layers

The model with three hidden layers of sizes [128, 64, 32] performed the best among the tested configurations. It achieved the lowest values for test RMSE, test MSE, and test MAE, indicating better predictive accuracy and generalization. Compared to models with fewer layers, this configuration likely captures more complex patterns in the data while avoiding overfitting.

✓ Different batch sizes

The comparison of different batch sizes shows that a batch size of 100 achieved the best performance, with the lowest test RMSE, test MSE, and test MAE values. As the batch size increases to 500 and 1000, the performance slightly worsens. This suggests that smaller batch sizes may help the model generalize better by introducing more gradient updates per epoch, leading to improved convergence. However, larger batch sizes might still be preferable in scenarios where training efficiency and stability are prioritized.

✓ Different learning rate

The comparison of different learning rates shows that a learning rate of 0.001 achieved the best performance across all metrics (test RMSE, test MSE, and test MAE). On the other hand, a very small learning rate (0.0001) also underperformed, potentially due to slow convergence. These results highlight the importance of choosing an optimal learning rate to balance convergence speed and model accuracy.

✓ Different activation functions

The best activation functions were ReLU and Leaky ReLU, while the worst was the sigmoid function. Sigmoid is not suitable for our solution because its output range is from 0 to 1, not -1 to 1. This limited range can cause issues like vanishing gradients, making it harder for deep networks to learn effectively. In contrast, ReLU and Leaky ReLU help mitigate these problems by allowing better gradient flow during training.

Comparison of Adam optimiser versus different state-of-the-art

optimisers

To compare Adam with other state-of-the-art optimisers we devised 2 experiments.

✓ With dropout

We used dropout of 50% which is the same value that was used in the paper introducing Adam.

We can see that with learning rate of 0.01 Adam was the best performing model. With learning rate of 0.001 RMSProp comes out on top. Lower learning rate seems to have inhibited the models ability to converge, which can most plainly be seen on the difference in Adagrad performance.

✓ Without dropout

When comparing the optimiser performance without the added dropout Adam came out on top. The only difference came out when comparing with added dropout, where RMSProp was slightly better. At the same time, as in the paper, Adagrad performed worst for us in the dropout model.

✓ Model results

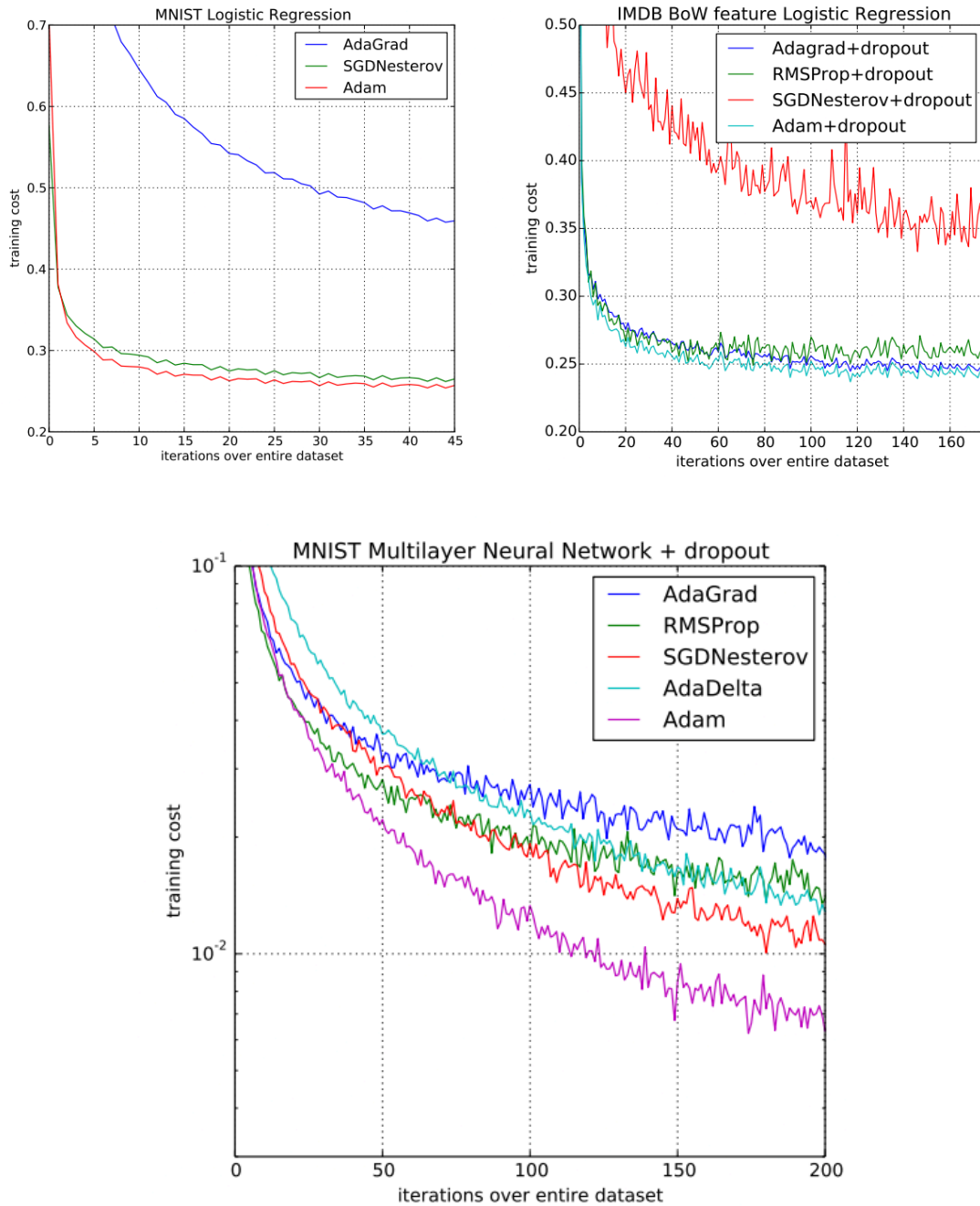
Parameters:

- ReLU activation
- batch size: 1000
- epochs: 100
- hidden layers: [128, 64, 32]
- learning rate: 0.001
- optimizer: Adam

✓ Comparison with paper

We focused on the Adam optimizer and compared it with several other optimization methods, as discussed in the article. Specifically, we utilized various techniques, including Adam, NAdamW, RMSProp, SGD, and AdaGrad. Our solution addresses a regression problem aimed at predicting house prices using a Multi-Layer Perceptron (MLP).

Results from the paper:



Source: <https://arxiv.org/abs/1412.6980>

In the paper, the authors compared multiple optimizers such as Adam, RMSProp, AdaGrad, and SGD with Nesterov momentum. The results indicated that Adam is the best choice among the optimizers mentioned. Similarly, our solution employed Adam, RMSProp, SGD, and AdaGrad, while also incorporating the NAdamW variant of Adam. Our test results confirm that Adam has the best performance of the dataset without dropout layer. Our experiments also show that

Adam has great performance with dropout applied. However we also discovered that this performance is dependant on the learning rate used. Additionally our experiments also confirm AdaGrad is the weakest optimiser once dropout is applied.

Overall our experiments confirm the papers conclusion that Adam is a good general purpose optimiser with best or at least very good performance both with and without dropout.

https://wandb.ai/xmagyarova-stu/NN_assignment1/reports/Neural-networks-Project-1--VmIldzoxMTk3NTEzOA